Handbook of Biometrics: Spoof Detection Scheme

Adaptive Normalized Representation Learning for Generalizable Face Anti-Spoofing

Le Truong Thinh (23127018)
Le Minh Duc (23127018)

University of Science, VNU-HCM
Faculty of Information Technology
Computer Science Department

Instructors:
Vo Thanh Lam
Nguyen Thanh Tinh

Ngày 19 tháng 4 năm 2026

Contents

1. Running the original code
   - Code repository and reproduction
   - Datasets
   - Experimental setup
2. Source code implementation of core technical components

Running the original code

Implementation Overview

Main Goal

Reproduce the official ANRL pipeline for generalizable face anti-spoofing.

- **Repository:** Hosted on TFace (Tencent's official face research suite).
- **Core Components Included:**
  - Adaptive Feature Normalization Module (AFNM)
  - Dual Calibrated Constraints (DCC)
  - Meta-learning optimization routine
- **License/Access:** Open-source via GitHub.

Technical Prerequisite

Requires PRNet for generating 3D-based facial depth maps.

Reproduction Roadmap: Setup

Initial Environment Preparation

1. **Clone Repository:**

Command

```
git clone https://github.com/Tencent/TFace.git
```

2. **Install Dependencies:**

Python Packages

```
PyTorch, OmegaConf, OpenCV, scipy, wandb
```

- **Next:** Configuring the PRNet depth estimator.

Reproduction Roadmap: Models & Data

3. **PRNet Checkpoint:** Place `prnet.pth` in `checkpoints/` directory.
4. **Download Datasets:**
   - CASIA-FASD, MSU-MFSD (Source)
   - NUAA (Unseen Target)
5. **Pre-process Depth Maps:**
   - Live samples: PRNet-generated geometry.
   - Spoof samples: Zero-depth maps (paper protocol).

Execution: Training and Testing

Core Training Pipeline

- **Main Training Command:**

```
python train.py -c configs/CM2N.yaml
```

- **Resuming Execution:**
  - `-resume /path/to/ckpt.tar`
- **Independent Testing:**
  - `-test` flag to evaluate on target domain (NUAA).

Logs & Monitoring

Offline logging via **Weights & Biases** (wandb.mode: offline).

Execution Environment

Hardware Specifications

All experiments conducted on **Google Colab**:

- **GPU:** NVIDIA T4 (15 GB VRAM)
- **RAM:** 12.7 GB System RAM
- **Runtime:** Python 3.10

- Sufficient for custom CNN backbone.
- Supports mixed-precision training.
- Batch size limited to 4 by VRAM constraints.

Example config: Datasets - Source Training Domains

CASIA-FASD

- **Subjects:** 12
- **Attacks:**
  - Low/High quality print
  - Video replay
- Role: Meta-train source

MSU-MFSD

- **Subjects:** 35
- **Attacks:**
  - Printed photo
  - Mobile phone replay
- Role: Meta-train + Val

Example config: Datasets - Unseen Target Domain

NUAA Database

- Frontal-face photograph spoofing database.
- Evaluation Role: Exclusively used as the **Unseen Target Domain** to test generalization.

- **Split Protocol:** `[CASIA + MSU]` $\rightarrow$ `[NUAA]`
- Shuffled domains each epoch (e.g CASIA as meta-train domain, MSU as meta-test domain or otherway) to simulate learning out-of-distribution.

Depth Supervision Protocol

Leveraging Geometric Cues for Spoof Detection

Sample Type	Depth Map Strategy
Live Faces	PRNet-generated geometry (32×32)
Spoof Attacks	Zero-depth maps (assumes plane surface)

- **Pre-trained model:** PRNet preserves facial geometry for real faces.
- **Rationale:** Spoof attacks (print, video) lack 3D depth, making zero-depth a strong discriminative signal.

Experimental Setup: Hyperparameters

Category	Parameter	Value
Optimizer	Algorithm	Adam
	Learning Rate	1×10^{-4}
	Weight Decay	1×10^{-5}
Scheduler	StepLR	Step 20, $\gamma = 0.5$
Training	Effective Batch	32 (4×8 accumulation)
	Meta Batch / Step Size	1 / 1×10^{-4}

- **Adam Optimizer:** Use to update weights in outer meta-learning loop.
- **StepLR Scheduler:** Moderate decay ($\gamma = 0.5$), make learning slower and more stable in last epochs.

Training Dynamics: Gradient Accumulation

Hardware Constraint Mitigation

Effective Batch Size = 32 achieved via **Gradient Accumulation**.

- **Problem:** T4 GPU VRAM limits per-step batch size to **4**.
- **Solution:** Accumulate gradients over **8 steps** before updating weights.
- Benefit: Preserves regularization behavior of large-batch training without exceeding memory limits.

Meta-learning Separation

Inner-loop (AFNM module - soft attention weights) updated with **meta-step size** (10^{-4}), separate from outer-loop base model update.

Hyperparameters: Loss weights rationale

Loss funcction

- **Depth Regression (0.1):** Auxiliary regularization.
- **DCC Losses (0.001):**
  - IDC: Inter-Domain Compatible
  - ICS: Inter-Class Separable
- **Note:** These act as regularization coefficients.

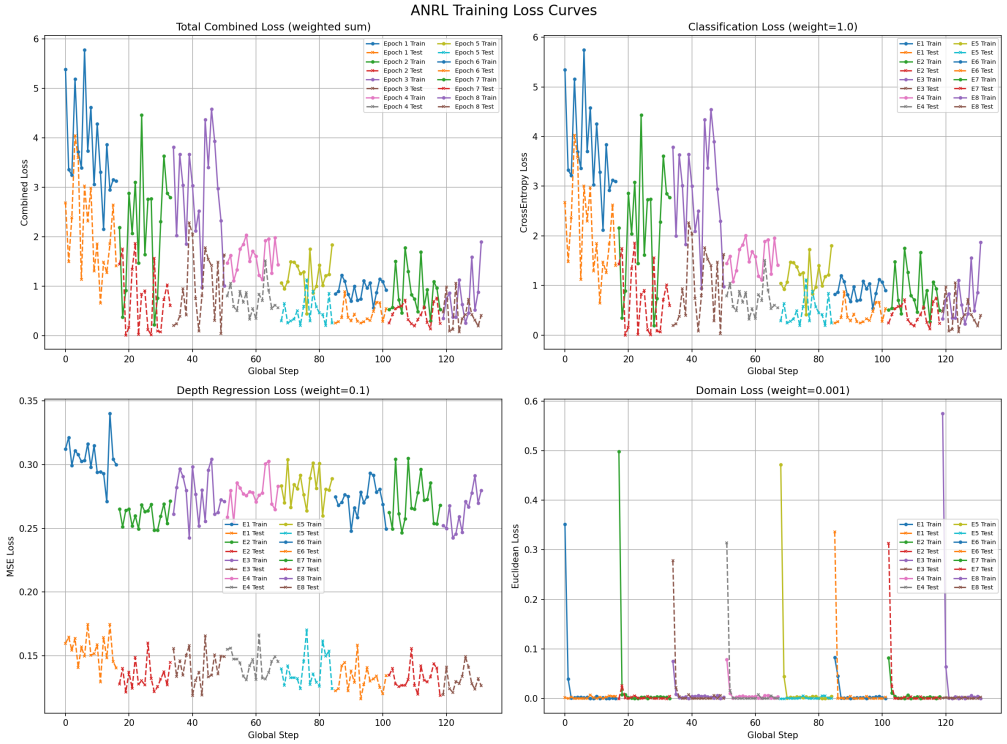

Figure: ANRL Training loss curves

$$\mathcal{L} = \mathcal{L}_{cls} + 0.1\mathcal{L}_{dep} + 0.001(\mathcal{L}_{IDC} + \mathcal{L}_{ICS})$$

ANRL Framework Architecture

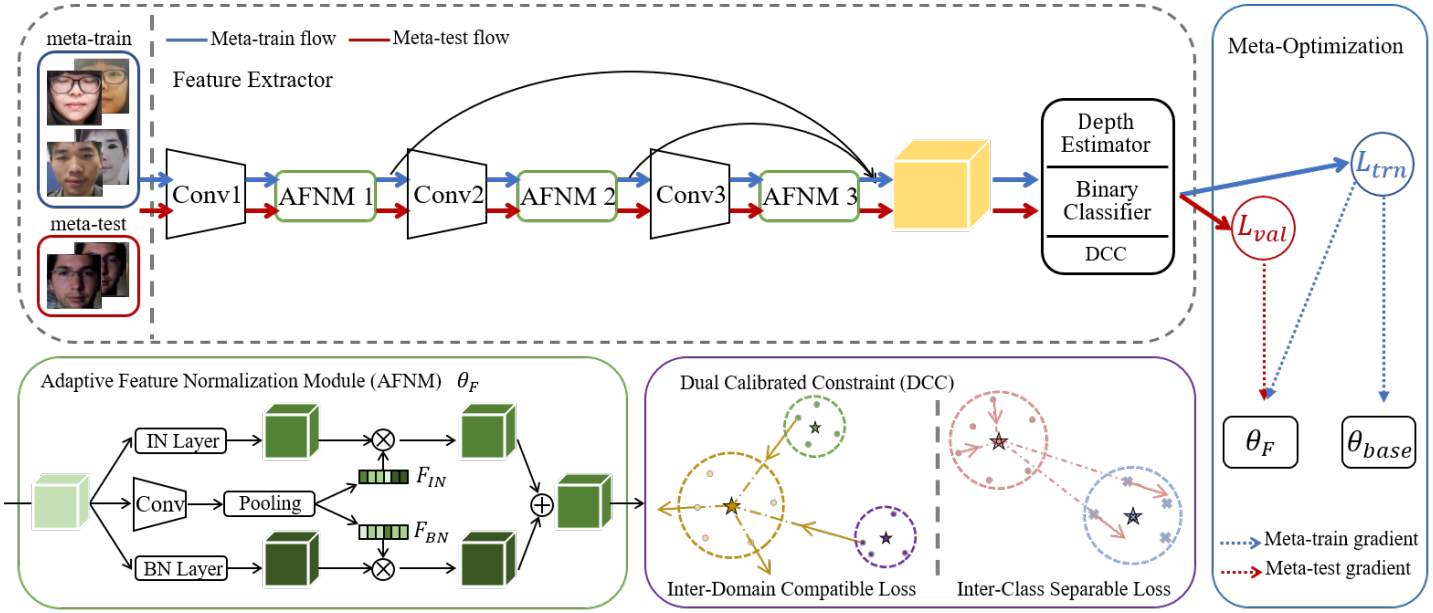

Figure: Overall ANRL Framework (Fig 2)

- **CNN Backbone:** Located at the center as the shared Feature Extractor.
- **Dual Pathways:** Integrates AFNM for both meta-train and meta-test episodes.

Technical Detail: Custom CNN Backbone

Stage	Layer	Channels/Stride	Output Size
Input	Image	6 (RGB+HSV)	256
Block 1	conv1-1 to conv1-4	64–196 / 1	256
	pool1-1	- / 2	128
Block 2	conv1-5 to conv1-7	128–196 / 1	128
	pool1-2	- / 2	64
Block 3	conv1-8 to conv1-10	128–196 / 1	64
	pool1-3	- / 2	32

- **Architecture:** 10-layer CNN backbone optimized for Colab T4 memory.
- **Note:** Connects directly to the Meta-Learner and Depth Estimator branches.

Evaluation Protocol & Performance Metrics

Primary Metrics

- **AUC (Area Under Curve):** Evaluates the model's ability to rank real vs. spoof samples correctly across all thresholds.
- **HTER (Half Total Error Rate):** The average of False Rejection Rate (FRR) and False Acceptance Rate (FAR).

- **Goal:** Achieving Higher AUC and Lower HTER translates to better cross-domain generalization.
- **Note:** t-SNE analysis is used to visualize feature separation between classes and domains.

Source code implementation of core technical components

Section 5.4: Source code implementation of core technical components [5]

1. Problem Definition & Depth Supervision

Geometric Depth Map Supervision [13]

- **Rationale:** Forces the network to learn geometric structure (real faces are 3D) rather than relying on texture biases from 2D spoofs.
- **Code Identification:** `src/datasets/DG_dataset.py` (Lines 60–62: setting depth to 0 for spoof labels).
- **Consequence:** Removing supervision leads to generalization failure; network "cheats" by focusing on high-frequency domain noise.

High-Dimensional Input Selection (RGB+HSV)

Multi-Channel Rationale [2]

- **Benefit:** HSV provides illumination (Value) and saturation signals robust to hardware-specific signatures.
- **Code Identification:** `src/datasets/DG_dataset.py` (`img_mode='rgb_hsv'`).
- **Consequence:** RGB-only models increase vulnerability to brightness fluctuations [13], yielding higher ACER/HTER during cross-dataset testing.

2. AFNM: Adaptive Feature Normalization Module

BN vs IN Trade-off [5]

- **Rationale:** BN is discriminative but sensitive to domain gaps; IN is stable but discards FAS-relevant style info.
- **Code Identification:** `AttentionNet` in `src/models/base_block.py` (Line 54).
- **Consequence:** Without AFNM, pure BN baseline HTER is 23.37% vs 15.67% for ANRL (I&C&M $\rightarrow$ O task).

- **Principle:** Balance factor α_c per-sample: $Y_c = \alpha_c X_c^{BN} + (1 - \alpha_c) X_c^{IN}$.

3. DCC: Dual Calibrated Constraints

IDC and ICS Loss Implementation

- **Identification:** `src/train.py` (Lines 183–212). DCC uses exponential moving average (EMA) centroids (momentum 0.9).
- **Rationale:** IDC aligns domain representations; ICS increases inter-class separation.
- **Consequence:** Removing DCC results in an absolute HTER increase of 2–4% (Table 3 in paper).

4. Meta-learning training Strategy

Simulation of Domain Shift

- **Steps:** 1. Inner Loop (Soft attention weights), 2. Meta-test, 3. Meta-Optimization.
- **Identification:** `src/train.py` (Lines 329–411).
- **Rationale:** Enables AttentionNet to learn α estimation that generalizes to novel distributions.
- **Consequence:** HTER climbs from 17.61% to 19.23% without meta-learning.

5. Network Architecture

- **FeatExtractor:** Integrates backbone with AFNM modules for domain-invariant extraction.
- **FeatEmbedder:** Discriminative head mapping features to class embeddings via ICS loss.
- **DepthEstimator:** Auxiliary head specialized in 3D geometric reconstruction.

Architecture Selection Rationale

Modular framework decouples feature learning from classification and regression for stable shared learning.

6. Summary: Paper $\leftrightarrow$ Code Mapping

Technique (Paper)	**Code Component**	**Consequence of Removal**
Depth Supervision	`DepthEstimator` (MSE)	Overfitting to domain noise
High-Dim Input	`rgb_hsv` 6-ch	Low robustness to hardware [2]
AFNM (BN+IN)	`AdapNorm` Logic	HTER increase +7.7%
DCC Losses	`loss_domain`, `loss_discri`	Loss of domain alignment
Meta-learning	`train()` Outer Loop	Higher HTER +1.62%

References

[2] Z. Boulkenafet, J. Komulainen, and A. Hadid. face anti-spoofing based on color texture analysis. CoRR, abs/1511.06316, 2015.

[5] H. et al. Joint physical-digital augmentation for face anti-spoofing, 2024.

[13] Y. Liu, A. Jourabloo, and X. Liu. Learning deep models for face anti-spoofing: Binary or auxiliary supervision. CoRR, abs/1803.11097, 2018.